

MASTER DIRECTIVE — COMPLETE REDEVELOPMENT OF THE STUDY PLATFORM

READ THIS ENTIRE PROMPT BEFORE CHANGING ANY CODE

You are taking ownership of the **overall redevelopment of an existing education application**.

This is NOT a simple UI redesign.

This is NOT a color/theme change.

This is NOT a request to add a few animations.

This is a **full product-level audit, UX redesign, UI redesign, architecture improvement, logic improvement, performance improvement and progressive code redevelopment**.

Your responsibility is to transform the existing application into a **premium, extremely polished, intelligent, minimalist and highly usable study platform**.

Think about the level of care, simplicity, consistency and product discipline associated with Apple's best products.

Do NOT copy Apple.

Instead, adopt the underlying philosophy:

Make the technology extremely sophisticated while making the experience extremely simple.

0. YOUR ROLE

Act simultaneously as:

- Principal Product Designer
- Senior UX Architect
- Senior UI Engineer
- Senior React/TypeScript Engineer
- Software Architect
- Performance Engineer
- Accessibility Engineer
- QA Engineer
- Product Strategist

You are not merely executing instructions.

You are expected to **identify problems yourself and solve them**.

If you discover a better approach than something explicitly described in this prompt, use your professional judgment.

Do not ask for permission for every obvious improvement.

1. PRIMARY OBJECTIVE

Redevelop the ENTIRE APPLICATION EXPERIENCE.

The final product should feel:

- premium
- calm
- intelligent
- minimal
- extremely polished
- fast
- responsive
- intuitive
- trustworthy
- modern
- cohesive

It should NOT feel like:

- an AI-generated dashboard
- a generic SaaS template
- a traditional EdTech website
- a collection of unrelated features
- a dashboard filled with cards
- a visually noisy application

The desired feeling is:

"Everything I need is here, but nothing is getting in my way."

2. VERY IMPORTANT — DO NOT REDESIGN BLINDLY

Before modifying the application, inspect the entire repository.

Do a complete technical and product audit.

Inspect:

- package.json
- source tree
- React components
- routes
- API routes
- server code
- public assets
 - `public/jee-cbt.html`
- styles
- hooks
- utilities
- Supabase migrations
- database assumptions
- authentication
- local storage
- planner logic
- YouTube/StudyTube logic
- question systems
- PYQ systems
- test engine
- analytics
- reminders
- focus functionality
- Android implementation
- build configuration
- deployment configuration
- environment variables
- error handling
- loading states
- responsive implementation

Trace important functionality from:

USER ACTION

↓

UI

↓

STATE

↓

BUSINESS LOGIC

↓

API



DATABASE / EXTERNAL SERVICE



RESULT

Do not assume anything.

3. CREATE AN INTERNAL PRODUCT AUDIT BEFORE REBUILDING

Before implementation, internally classify the existing product into:

KEEP

Good functionality that should remain.

IMPROVE

Useful functionality with poor UX or implementation.

REFACTOR

Functionality whose architecture is unnecessarily complicated.

REBUILD

Features whose current implementation prevents a good experience.

REMOVE

Redundant, confusing or unnecessary functionality.

ADD

Capabilities required to make the overall product coherent.

Do not preserve bad UX simply because it already exists.

Do not delete good functionality simply because the UI is being redesigned.

4. GOLDEN RULE

PRESERVE THE FUNCTIONAL ENGINE.

REBUILD THE EXPERIENCE.

If the current logic works well, preserve it.

If the architecture is bad, improve it.

If something is broken, fix the root cause.

If something is duplicated, consolidate it.

If something is unnecessarily complicated, simplify it.

Do NOT create parallel duplicate systems.

5. IMPORTANT ARCHITECTURE ISSUE

The project contains a very large `public/jee-cbt.html` application alongside a React/TanStack application structure.

Treat this as an architectural concern that requires investigation.

Do not automatically delete it.

Instead determine:

- which functionality lives inside it
- which functionality should migrate
- what can safely be extracted
- what should become React components
- what should become hooks
- what should become services
- what should become API modules
- what should remain static
- what should remain in the public layer temporarily

If progressive migration is safer, do progressive migration.

If a complete replacement is genuinely safer, do it carefully.

Do not perform a destructive rewrite without understanding dependencies.

6. PRODUCT PRINCIPLE

The app can contain enormous internal complexity.

The student should not experience that complexity.

Think:

SYSTEM

AI + data + planner + analytics + videos + questions + performance + scheduling

↓

SIMPLE EXPERIENCE

What should I do now?

That is the fundamental product principle.

7. THE NEW INFORMATION ARCHITECTURE

Simplify the primary experience.

Recommended primary navigation:

Home

What should I do now?

Plan

What should I study today and this week?

StudyTube

What should I watch/learn?

Practice

How should I practice?

Progress

How am I improving?

Everything else should be contextual or secondary.

Do not expose every feature as a top-level navigation item.

8. DESIGN SYSTEM — BUILD FIRST

Before redesigning individual screens, establish a unified design system.

Create reusable primitives for:

- buttons
- inputs
- selects
- dialogs
- sheets
- tabs
- navigation
- cards
- lists
- menus
- tooltips
- progress indicators
- badges
- avatars
- skeletons
- alerts
- empty states
- error states

Establish:

- typography scale
- spacing scale
- radius system
- elevation system
- border system
- color tokens
- animation tokens
- responsive breakpoints

Do not allow individual screens to invent their own visual language.

9. VISUAL PHILOSOPHY

Use a restrained visual system.

Prefer:

- neutral surfaces
- excellent whitespace
- strong typography
- subtle borders
- subtle elevation
- one primary accent
- carefully used semantic colors
- precise alignment

Avoid:

- rainbow interfaces
- excessive gradients
- excessive glassmorphism
- giant shadows
- excessive rounded cards
- random colored containers
- decorative UI
- unnecessary illustrations
- excessive badges
- excessive icons

10. MINIMALISM DOES NOT MEAN EMPTY

The application must NOT become boring.

The goal is:

MINIMAL INTERFACE

+

RICH INTERACTION

Use sophistication through:

- contextual recommendations
- intelligent state changes
- adaptive content
- useful transitions
- progressive disclosure
- responsive interactions
- smart search
- meaningful progress
- AI assistance

Do NOT use visual noise to create excitement.

11. DASHBOARD — COMPLETE REDEVELOPMENT

The Dashboard is the most important screen.

Do not treat it as an analytics dashboard.

It should be the student's:

TODAY SCREEN

The first question it answers:

What should I do right now?

12. NEXT BEST ACTION

Create a system-level concept called:

NEXT BEST ACTION

The application should determine the most useful next activity.

Possible results:

- Continue unfinished lesson
- Revise weak topic
- Practice questions
- Take scheduled test
- Watch recommended StudyTube lesson
- Complete overdue revision
- Start today's planned session

The UI should prominently surface ONE primary action.

Do not make the student choose between 15 equally important buttons.

13. DASHBOARD STRUCTURE

Suggested hierarchy:

Greeting/context

Small and calm.

Next Best Action

Dominant.

Today's Plan

Timeline/list.

Progress Snapshot

Minimal.

AI Insight

Contextual.

Upcoming

Only if genuinely useful.

The exact layout may be changed if your audit reveals a better structure.

14. DASHBOARD MUST BE PERSONALIZED

The Dashboard should change based on state.

No study plan

Show:

Create your study plan.

Plan exists

Show today's plan.

Unfinished session

Show resume.

Weak topic

Surface practice.

Upcoming test

Prioritize preparation.

Student completed today's work

Celebrate subtly and recommend the next useful activity.

This should be logic-driven.

15. AI PLANNER — REBUILD AS A CORE PRODUCT

The AI Planner must NOT feel like a form.

It should feel like an intelligent planning system.

Use progressive onboarding.

Collect only information necessary to build a useful plan.

Examples:

- target exam
- target date
- available time
- current level
- subject confidence
- existing progress
- preferred learning style
- upcoming commitments

Then combine that with existing application data.

16. AI PLANNER SHOULD USE REAL DATA

Where available, use:

- test results
- question accuracy
- topic mastery
- study history
- video completion
- revision history
- missed sessions
- available time
- syllabus structure

Do not generate generic schedules.

The plan should be personalized.

17. AI PLANNER MUST EXPLAIN ITSELF

When generating a plan, explain important decisions.

Example:

Physics has been given additional practice time because your recent accuracy is lower than your other subjects.

This creates trust.

18. ADAPTIVE PLANNING

The plan must not be static.

If a student misses a session:

Recalculate intelligently.

Do not simply push everything forward.

Consider:

- remaining time
- priority
- exam proximity
- topic difficulty
- previous performance
- fatigue/load

Then suggest a realistic adjustment.

19. STUDYTUBE — COMPLETE REDEVELOPMENT

StudyTube should NOT simply be:

YouTube embedded inside the application.

It should become:

A STUDY-FIRST VIDEO PLATFORM.

Familiar enough to understand immediately.

Different enough to eliminate unnecessary distractions.

20. STUDYTUBE HOME

Prioritize:

Continue Learning

Recommended for Your Plan

Weak Topics

Revision

Saved

Recently Studied

Do not create endless recommendation shelves.

Every section must justify its existence.

21. STUDYTUBE RECOMMENDATION ENGINE

Where technically possible, connect recommendations to:

- syllabus
- target exam
- AI Planner
- weak topics
- test performance
- previous viewing
- video completion
- saved content
- learning sequence

Optimize for:

LEARNING OUTCOME

not:

WATCH TIME

22. STUDYTUBE VIDEO CARD

Every card should answer:

- What is this?
- Is it relevant to me?
- How long is it?
- How much have I watched?

Example:

Gauss Law — Electrostatics

Physics · JEE Main
42 min · Intermediate

82% complete

Keep metadata subtle.

23. STUDYTUBE VIDEO PAGE

The learning flow should become:

WATCH



UNDERSTAND



NOTES



PRACTICE



NEXT

Include where technically possible:

- video
 - chapters
 - progress
 - transcript
 - AI notes
 - key concepts
 - bookmarks
 - Ask AI
 - quiz
 - practice
 - next recommended lesson
-

24. VIDEO RESUME

Always remember:

- last position
- completion percentage
- notes
- bookmarks

Make:

Resume at 18:32

an important interaction.

25. CONTEXTUAL AI

AI should understand where the student is.

Examples:

From StudyTube:

Explain this concept.

From Planner:

Why did you schedule this?

From Progress:

Why am I weak here?

From Practice:

Explain my mistake.

From Dashboard:

What should I do now?

AI should feel like one intelligence layer across the product.

26. PRACTICE

Practice should become outcome-driven.

Primary options:

- Continue
- Smart Practice
- Weak Topics
- PYQs
- Speed Drill
- Tests

Do not force students through complicated menus.

27. TEST/CBT

Keep the exam experience serious.

Prioritize:

- question
- answer
- timer
- navigation
- review
- submission

No unnecessary decorative elements.

Test reliability is more important than visual experimentation.

28. PROGRESS

Reframe analytics around:

Am I getting better?

Show:

- mastery
- accuracy
- speed
- consistency
- weak topics
- mistake patterns
- improvement

Then immediately provide:

WHAT SHOULD I DO NEXT?

Analytics should lead directly into action.

29. CONNECT THE SYSTEMS

This is extremely important.

Do not leave features isolated.

Create logical connections:

Test result → weak topic

Weak topic → StudyTube recommendation

StudyTube completion → practice recommendation

Practice result → mastery update

Mastery update → planner adjustment

Planner adjustment → dashboard

This creates a true learning system.

30. ONE SOURCE OF TRUTH

Avoid multiple competing states.

For important student information, establish clear ownership.

Examples:

- study progress
- video progress
- planner state
- mastery
- test results
- user preferences

Avoid storing the same truth independently in several unrelated places.

31. DATA CONSISTENCY

Whenever a user action changes something important, update dependent systems appropriately.

Example:

Student completes a planned lesson.

Potential updates:

- planner
- dashboard
- progress
- study history
- recommendation system

Do not leave the UI showing stale information.

32. PERFORMANCE

Audit:

- rendering
- state updates
- DOM size
- API requests
- duplicate requests
- expensive calculations
- video embeds

- images
- event listeners
- animations
- unnecessary dependencies

Use:

- lazy loading
- caching
- memoization where justified
- debouncing
- efficient queries
- skeleton loading

Do not optimize blindly.

Measure logical bottlenecks.

33. RESPONSIVE DESIGN

Desktop, tablet and mobile are different experiences.

Design intentionally for:

Desktop

High information efficiency.

Tablet

Balanced.

Mobile

Focused and touch-friendly.

Do not simply shrink desktop.

34. ACCESSIBILITY

Implement:

- keyboard navigation
- focus states
- semantic HTML
- appropriate ARIA
- contrast

- touch target sizes
- readable text
- screen-reader labels

Accessibility is part of product quality.

35. LOADING STATES

Never leave users staring at blank screens.

Use contextual skeleton/loading states.

AI loading should communicate meaningful progress.

Example:

Analyzing your recent performance...

not:

Loading...

36. ERROR STATES

Never expose raw technical errors.

Example:

We couldn't load your StudyTube recommendations.

[Try Again]

[Continue with Saved Content]

Log technical details internally.

37. EMPTY STATES

Every empty screen should explain:

1. What is empty?
 2. Why?
 3. What should the user do?
-

38. MICRO-INTERACTIONS

Use motion sparingly.

Motion should communicate:

- progress
- completion
- navigation
- state changes
- hierarchy

Never animate simply because you can.

39. DESIGN REVIEW FOR EVERY SCREEN

Before declaring any screen complete, ask:

What is the primary task?

Is it obvious?

What can be removed?

Is anything competing with the primary task?

Are there unnecessary cards?

Are there unnecessary buttons?

Are there unnecessary colors?

Does the screen work without explanation?

Is the interaction predictable?

Does it feel premium?

40. APPLE-LEVEL QUALITY BAR

Do NOT imitate Apple's appearance.

Imitate the discipline.

Every element must earn its place.

Every interaction must have a reason.

Every transition must have a purpose.

Every piece of information must have hierarchy.

Every error must have recovery.

Every important action must have feedback.

Every feature must belong to the overall product.

41. PROACTIVE IMPROVEMENTS

You are explicitly authorized to identify and implement improvements not listed in this prompt.

If you discover:

- a better navigation model
- better information architecture
- better planner interaction
- better StudyTube flow
- better data architecture
- better state management
- better performance solution
- missing UX states
- accessibility problems
- broken logic
- duplicated logic
- technical debt
- inconsistent components
- confusing workflows

you should fix them.

Do not limit yourself to the exact examples in this prompt.

Use professional judgment.

42. BUT DO NOT INVENT RANDOM FEATURES

Proactive improvement does NOT mean adding features for the sake of features.

Every new feature must answer:

Does this materially improve the student's learning experience?

If not, don't add it.

43. CODE QUALITY

Improve code quality progressively.

Look for:

- giant components
- duplicated code
- duplicated state
- magic values
- dead code
- unused imports
- inconsistent naming
- repeated API logic
- inline styles
- unnecessary global state
- fragile DOM manipulation
- unnecessary dependencies

Do not refactor purely for aesthetics.

Prioritize changes that improve:

- reliability
 - maintainability
 - performance
 - testability
 - future development speed
-

44. SECURITY

Review:

- exposed secrets
- environment variables
- API keys
- client-side trust
- authentication
- authorization
- Supabase policies
- unsafe input
- API validation

Never expose secrets in frontend code.

Do not weaken existing security while redesigning.

45. DO NOT BREAK DEPLOYMENT

After changes verify:

- development build
- production build
- routes
- APIs
- PWA
- Android integration where applicable
- environment configuration

Do not leave the project in a visually improved but undeployable state.

46. QA — DO NOT ASSUME IT WORKS

After implementation, test systematically.

Test:

Authentication

Dashboard

Planner

StudyTube

Video progress

Practice

Tests

PYQs

Analytics

AI interactions

Reminders

Focus features

Mobile

Desktop

Error states

Empty states

Loading states

47. REGRESSION TESTING

Before finishing, verify that previously working functionality still works.

Pay special attention to:

- test submission
- question navigation
- timers
- planner persistence
- video progress
- authentication
- database writes
- API calls
- reminders

- Android functionality

A redesign that breaks existing functionality is a failed redesign.

48. BUILD VALIDATION

Run the appropriate:

- lint
- type checks
- tests
- build
- relevant validation scripts

Fix errors.

Do not finish with known compilation errors.

Do not hide errors by suppressing them unless there is a legitimate reason.

49. FINAL USER JOURNEYS

Validate complete journeys.

JOURNEY 1

New student

→ onboarding

→ AI planner

→ generated plan

→ dashboard

→ first study session

JOURNEY 2

Dashboard

→ Next Best Action

→ StudyTube

→ video

→ notes

→ practice

→ updated progress

JOURNEY 3

Test

→ results

→ weak topic

→ recommended StudyTube lesson

→ practice

→ planner adjustment

JOURNEY 4

Missed session

→ planner detects it

→ AI adjusts schedule

→ dashboard updates

JOURNEY 5

Student asks AI:

"Why am I weak in Electrostatics?"

AI

→ analyzes relevant data

→ explains

→ recommends action

→ optionally adds action to plan

50. FINAL "10 SECOND" TEST

A student opening Home should understand within roughly 10 seconds:

1. Where am I?
2. What should I do now?
3. Why?
4. How long will it take?
5. What happens afterward?

If not, redesign.

51. FINAL "APPLE" TEST

For every major screen:

Can something be removed?

Remove it.

Can something be simplified?

Simplify it.

Can two actions become one?

Combine them.

Is the primary action obvious?

If not, fix hierarchy.

Is advanced functionality cluttering the interface?

Move it behind progressive disclosure.

Does the system remain powerful after simplification?

If yes, you succeeded.

52. MOST IMPORTANT PRODUCT RULE

Never optimize for:

"How many features can the application display?"

Optimize for:

"How effortlessly can the student accomplish the next useful thing?"

53. DEFINITION OF DONE

Do NOT declare the project complete merely because:

- the UI looks better
- the colors changed
- the dashboard looks modern
- the code compiles

The project is complete only when:

PRODUCT

The application feels like one coherent product.

UX

Important actions are obvious.

UI

The visual system is consistent and premium.

DASHBOARD

It intelligently answers "What should I do now?"

AI PLANNER

It genuinely adapts to the student.

STUDYTUBE

It feels like a real study-first video platform.

PRACTICE

It connects to weaknesses and learning goals.

PROGRESS

It leads to useful actions.

AI

It understands context across the application.

ARCHITECTURE

The codebase is moving toward clean separation of concerns.

PERFORMANCE

The application feels fast.

RESPONSIVENESS

Desktop and mobile both feel intentionally designed.

ACCESSIBILITY

Core workflows are accessible.

RELIABILITY

Existing functionality continues to work.

QUALITY

There are no obvious broken interactions, dead buttons or unfinished states.

54. FINAL DIRECTIVE

Do not treat this as:

"Make my existing website prettier."

Treat it as:

"Rebuild this product into the best version of itself."

Study the existing code.

Understand the existing product.

Identify what is good.

Identify what is weak.

Fix the architecture where necessary.

Fix the logic where necessary.

Redesign the UX.

Redesign the UI.

Connect the systems.

Remove unnecessary complexity.

Preserve valuable functionality.

Add improvements when justified.

Test everything.

Then polish everything again.

The final result should have:

THE SIMPLICITY OF A GREAT CONSUMER PRODUCT

combined with

THE POWER OF A SOPHISTICATED LEARNING SYSTEM.

The complexity belongs inside the system.

The clarity belongs in front of the student.

DO NOT JUST MAKE IT LOOK BETTER.

MAKE IT WORK BETTER.

MAKE IT FEEL BETTER.

MAKE THE ENTIRE PRODUCT BETTER.